ORIGINAL RESEARCH FRAMEWORK | REPRODUCIBLE FEASIBILITY STUDY

Beyond Serialization: Evidence-Certified Graph Reasoning for Offline Gemma 4 Software Agents

Subtitle: From structure-aware graph representation to failure-conditioned code localization, adaptive context selection, and verifiable multi-hop reasoning

Author: Keedar Kale

Track: Google - The Gemma 4 Developer Agent Paper Track (2026)

Status: Original research framework and executable synthetic diagnostic. Gemma 4 inference, GNN training, and real-repository patch resolution are *not* claimed.

Abstract

Local coding agents face a structural bottleneck: a bug report can describe observable behavior while the faulty function lies several dependencies away, and serializing an entire repository graph wastes limited model context. Adapting an unpublished research agenda on graph-language-model representations and verified multi-hop reasoning to software engineering, we propose **Evidence-Certified Witness Graphs (ECWG)**. The design represents a repository as a typed graph; anchors retrieval at observed pre-fix test failures; supplies compact, inspectable dependency paths to a Gemma 4 patch generator; and checks claimed graph edges before prioritizing executable regression tests. It also specifies a falsifiable *representation-choice* study comparing text serialization, subgraph retrieval, GNN-derived features, and their hybrid under matched context and compute budgets. A standard-library prototype implements the static graph-localization and path-certificate portion. On 16 deliberately constructed, multi-file Python bugs, top-1 localization is 12/16 for failure-conditioned ranking versus 4/16 for a simple lexical baseline; top-2 localization is 16/16 versus 4/16. Removing test anchors reduces the graph method to 4/16; consistent identifier renaming leaves its top-1 result at 12/16. Sixteen genuine paths pass an edge-membership check and 16 intentionally corrupted paths fail. These diagnostics demonstrate feasibility and an important entrypoint-fault limitation, **not** state-of-the-art repair performance. We provide executable code and a pre-specified real-repository evaluation protocol.

1. Introduction and research questions

Code repositories are graphs: functions call other functions, modules import dependencies, tests exercise APIs, and edits propagate through callers. For a memory-constrained local agent, a flat edge list can be too long, a semantic nearest-neighbor search can favor descriptively named wrappers over faulty implementations, and a fluent explanation can invent dependencies. Prior research establishes that how graphs are serialized matters [2], that retrieved subgraphs can help graph question answering [3], and that graph navigation can assist code localization [4]. The remaining question is not simply whether graphs help, but **which representation is warranted, which evidence can be trusted, and whether that evidence changes executable repair success**.

This work transfers three questions from the author's unpublished PhD proposal, *Beyond Serialization: Structure-Aware Integration of Graphs and Large Language Models for Reliable Multi-Hop Reasoning*, into the narrower setting of offline software repair:

RQ1 - Representation. Can a short, order-independent, typed *witness subgraph* preserve code relationships more efficiently than full textual serialization?

RQ2 - Choice. Under which combinations of graph topology, issue-text quality, witness availability, and memory budget should an agent use text retrieval, graph paths, or GNN-derived representations?

RQ3 - Verification. Can machine-checked dependency certificates reduce invalid multi-hop claims, and does coupling them with real tests improve patch reliability?

The prototype answers a limited diagnostic portion of RQ1 and implements a basic certificate check for RQ3. RQ2 and downstream patch correctness remain hypotheses for evaluation, not established empirical findings.

2. Relation to prior work and intended novelty

Fatemi, Halcrow, and Perozzi [2] show that textual graph encodings are task- and structure-sensitive. LLaGA [5] adapts graph information into a language model's embedding space; G-Retriever [3] performs retrieval over

large textual graphs. In software engineering, LocAgent [4] already uses graph-guided multi-hop code localization. Therefore **graph-based retrieval, graph-to-LLM integration, and path reasoning are not novel by themselves**.

ECWG's research distinction is the *joint, falsifiable design*: (i) condition graph paths on actual failing-test witnesses rather than issue similarity alone; (ii) expose edge-level certificates and separate graph validity from causal validity; (iii) compare representation choices with controlled context budgets and an optional learned GNN; and (iv) link retrieval decisions to executable repair evidence while reporting when graph routing harms shallow faults. The current release validates only a subset of that combination. It neither reproduces nor claims to outperform the systems above.

3. Method: Evidence-Certified Witness Graphs

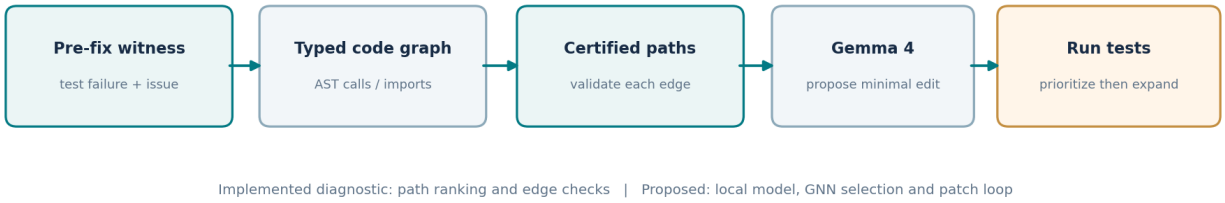


Figure 1. Evidence-certified pipeline. The pilot implements graph ranking and path checks; model integration is proposed.

3.1 Typed repository evidence

For repository R, construct a directed graph **G = (V, E, t)**, with nodes for tests and code symbols; edge types may include CALLS, IMPORTS, CONTAINS, INHERITS, and TESTS. Attach file spans and code text; where permitted, attach provided node embeddings. A failing test **f** observed *before* generating a patch is a witness anchor. For each candidate production function **v**, retrieve bounded paths **P(f,v)** along supported graph relations. A path is a **structural witness**, not proof that v caused the failure. When tests cannot run or their identities are unavailable, use issue-only search and explicitly label the fallback.

To make graph evidence robust to arbitrary listing order, normalize node identities and edge types, and serialize a selected subgraph in a canonical order. Report a short path such as `test_checkout -> checkout -> calculate_total`, with explicit, checkable edges and linked source spans. Do not expose reference fixes, post-fix test outputs, or evaluation labels to retrieval.

3.2 Measured localization prototype

The executable prototype parses Python abstract syntax trees to identify top-level functions and directly resolvable imports/calls. For each production symbol v, the elementary lexical score **L(v)** is issue/test token overlap against symbol names and docstrings, normalized across candidates. Let **R(v)** indicate reachability from the failing test within four call edges, and **D(v)** indicate a reachable terminal production symbol (no further observed local callee). The deliberately fixed experimental score is:

$$S(v) = 0.25\,L(v) + 0.20\,R(v) + 0.55\,D(v).$$

A terminal preference probes a specific hypothesis: lexical wrappers can conceal deeper bugs. It is *not* a learned diagnosis, and it predictably fails when a wrapper itself contains the error. Graph extraction is intentionally lightweight and misses reflection, dynamic dispatch, and cross-language edges.

3.3 Path certification and repair loop

A certificate is an ordered symbol list **pi = (v0,...,vk)** beginning at a witnessed test and ending at a proposed target. The deterministic checker accepts only if every node exists and each consecutive directed edge appears in the extracted graph. Invalid paths are rejected or regenerated. This check rules out invented **graph** edges; it does **not** prove runtime execution, fault causality, or semantic correctness.

The *proposed, not yet executed* Gemma 4 loop uses a context budget **B**: greedily select short source spans that add distinct witness/issue coverage per token, ask a single locally hosted Gemma 4 model for a minimal patch, run syntax/static checks and tests linked to edited symbols, and then run a broader regression suite. The agent must report unresolved or untested outcomes rather than label them successful. Graph-selected tests are a prioritization heuristic, never a replacement for broad validation.

3.4 A representation-choice study, not a presupposed GNN win

To implement RQ2, compare four settings on identical repository snapshots: **T** = text edge/adjacency serialization; **R** = retrieval of short typed witness subgraphs; **G** = GNN-encoded graph features with a lightweight projector; and **H** = retrieved subgraph followed by GNN encoding. Hold the Gemma 4 checkpoint, issue information, total prompt budget, and evaluation split fixed. Measure graph size, average degree, source-text descriptiveness, witness coverage, and available labels. Fit a tiny selector only on training repositories to choose among T/R/G/H under a latency and memory ceiling; compare it against always-R and oracle-free fixed policies. The GNN/projector and learned selector are **unimplemented** in this release. Their expected value is a testable question, especially on sparse-text or structurally complex code, not a reported result.

4. Executed diagnostic and results

We created **16 executable synthetic multi-file Python cases**. Each has a test, a descriptive API wrapper, a low-level numeric function, and an irrelevant legacy function. Twelve bugs are injected in the lower-level function; four are injected in the wrapper. Each assertion was verified to fail before a temporary known fix and to pass after it; the fix was then reverted. Ground-truth edit symbols were used only by the evaluator, not by localization. The depth distribution is intentionally biased toward deeper faults and **must not** be treated as representative of real software.

The lexical baseline and witness ranking receive the same issue and test identifier. A no-witness ablation removes the test identifier from graph routing. A source-preserving perturbation consistently renames the imported calculate function and call sites, leaving the fault type unchanged. For certificate sanity checks, the script reconstructs one graph path to each ground-truth symbol and replaces its last node with a nonadjacent real legacy symbol to create a bad certificate.

Synthetic diagnostic (16 cases)	Result
Lexical baseline, correct target at rank 1	4/16 (25%)
Witness-path ranking, correct target at rank 1	12/16 (75%)
Lexical baseline, target within top 2	4/16 (25%)
Witness-path ranking, target within top 2	16/16 (100%)
Witness ranking without test anchor, rank 1	4/16 (25%)
Witness ranking after identifier rewrite, rank 1	12/16 (75%)
Constructed genuine graph certificates accepted	16/16
Single-edge-tampered certificates rejected	16/16

Failure analysis. The witness score succeeds on all 12 engineered lower-level faults but fails to rank any of the four wrapper-level faults first. The lexical baseline has the opposite pattern. Removing the witness removes the graph method's top-1 benefit in this fixture design. The renaming probe checks one semantics-preserving identifier change, **not** general robustness to renaming or all edge orderings. Certificate checks are direct graph-membership unit tests, **not** evidence that an LLM can produce faithful or causally correct explanations. There are no trained GNN results, local Gemma 4 outputs, repository-scale metrics, or validated generated patches here.

5. Falsifiable evaluation on real developer tasks

The competition exposes public development tasks and code graphs/embeddings for reproducible offline experimentation [1, 6]. A rigorous next study should group held-out instances by repository and freeze choices before evaluating a held-out group. Reference fixes may be used for **offline scoring only**. Where failing tests exist, obtain pre-patch witnesses without using post-patch information; otherwise report an issue-only stratum separately. Use equivalent budgets (e.g., **2,048/4,096 input tokens**) for text, BM25, embedding search, graph-only diffusion, witness paths, T/R/G/H, and the adaptive selector. Do not compare a GNN trained with extra labels against a zero-shot model without accounting for that supervision.

Evaluate **(a)** file/function Recall@1/5 and MRR; **(b)** edge validity, path coverage, and invalid-claim rate; **(c)** actual resolved issues based on executable tests; **(d)** tokens, latency, peak GPU memory, and test costs; **(e)** selective abstention and confidence calibration. Test degradation at 1-6 hops, across graph sizes, and under identifier renaming, textual noise, edge-order permutations, missing edges, and shallow-vs-deep faults. Primary

ablations remove witnesses, path checks, GNN embeddings, and representation selection independently. Report paired confidence intervals, repository-level error slices, fixed seeds, exact model/harness versions, and negative outcomes. A feasible lightweight design uses a single supported local Gemma 4 variant and no cloud inference at test time.

Disconfirmation criteria: if R fails to beat matched-budget BM25 on held-out code, the supposed retrieval benefit does not generalize; if G adds cost without significant gains, the selector should prefer R/T; if path validity rises but patch resolution does not, certificates improve auditability rather than software correctness. A genuine improvement requires better real issue resolution or reliable reductions in compute at matched performance.

6. Limitations, open resources, and conclusion

The observed ranking gain is conditional on a tiny, artificial test set with deliberately favorable graph structure; it cannot establish superiority to LocAgent, dense embeddings, or learned code localization. The call graph is incomplete, the terminal heuristic is brittle, and a valid static path can still mislead. The code bundle contains the synthetic fixtures, scripts, per-case results, a certificate checker, and an issue-only graph adapter; it **does not** contain the proprietary/competition dataset, a trained GNN, a Gemma 4 model, or a patch-generation agent. The adapter expects a generic graph schema and requires validation against the actual Kaggle artifacts.

The central research claim is deliberately narrower than “graphs make coding agents better”: **graph evidence should be selected conditionally, preserved as short checkable paths, and judged by downstream executable outcomes**. ECWG supplies an inspectable first diagnostic and a directly falsifiable program for testing this claim with small offline coding models. The longer-term contribution is a decision rule for when graph structure merits its cost, not an assumed universal GNN advantage.

References

[1] Markowitz, E., et al. (2026). *Google - The Gemma 4 Developer Agent Paper Track*. Kaggle. https://www.kaggle.com/competitions/gemma-4-developer-agent-paper

[2] Fatemi, B., Halcrow, J., and Perozzi, B. (2024). *Talk like a Graph: Encoding Graphs for Large Language Models*. ICLR. https://proceedings.iclr.cc/paper_files/paper/2024/hash/bf72f65f30eedf5d48da6980ee02b589-Abstract-Conference.html

[3] He, X., et al. (2024). *G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering*. NeurIPS. https://proceedings.neurips.cc/paper_files/paper/2024/hash/efaf1c9726648c8ba363a5c927440529-Abstract-Conference.html

[4] Chen, Z., et al. (2025). *LocAgent: Graph-Guided LLM Agents for Code Localization*. ACL, pp. 8697-8727. https://aclanthology.org/2025.acl-long.426/

[5] Chen, R., et al. (2024). *LLaGA: Large Language and Graph Assistant*. ICML. https://proceedings.mlr.press/v235/chen24bh.html

[6] Google DeepMind / Kaggle (2026). *Getting Started - Gemma 4 Developer Agent* (competition data guide). https://www.kaggle.com/code/ryanholbrook/getting-started-gemma-4-developer-agent/input